# BLOQUE 1 — ¿Qué es programar?

1.1 ¿Qué es la programación?

- Qué significa programar
- Qué es un algoritmo
- Ejemplos sin código (vida real)

1.2 ¿Cómo piensa un ordenador?

- Entrada → Proceso → Salida
- Lógica básica
- Errores comunes de pensamiento

1.3 ¿Qué es Java?

- Historia breve
- Qué es la JVM
- Compilar vs ejecutar
- Por qué aprender Java

# BLOQUE 2 — Primer contacto con Java

2.1 Instalación y entorno

- JDK
- IDE (Eclipse / IntelliJ)
- Estructura de un proyecto

2.2 Nuestro primer programa

- Clase
- Método main
- System.out.println
- Errores típicos

# BLOQUE 3 — Variables y Tipos de Datos

3.1 ¿Qué es una variable?

- Concepto de memoria

3.2 Tipos primitivos

- int
- double
- boolean
- char

3.3 String y tipos no primitivos

3.4 Operadores

- Matemáticos
- Comparación
- Lógicos

# BLOQUE 4 — Control de Flujo

4.1 Condicionales

- if
- else
- else if
- switch

4.2 Bucles

- for
- while
- do-while

4.3 Buenas prácticas en estructuras de control

# BLOQUE 5 — Métodos (Funciones)

5.1 ¿Qué es un método?

5.2 Parámetros

5.3 Return

5.4 Métodos void

5.5 Modularización del código

# BLOQUE 6 — Arrays

6.1 ¿Qué es un array?

6.2 Recorrer arrays

6.3 Arrays multidimensionales

6.4 Errores comunes

# BLOQUE 7 — Introducción a la POO

*(Aquí empieza el verdadero cambio mental)*

7.1 ¿Qué es la Programación Orientada a Objetos?

- Diferencia con programación estructurada

7.2 Clases

7.3 Objetos

7.4 Atributos

7.5 Métodos de instancia

# BLOQUE 8 — Constructores

8.1 ¿Qué es un constructor?

8.2 Sobrecarga de constructores

8.3 this

# BLOQUE 9 — Encapsulamiento

9.1 Modificadores de acceso

- public
- private
- protected

9.2 Getters y Setters

9.3 Validación de datos

9.4 Buen diseño de clases

# BLOQUE 10 — Herencia

10.1 extends

10.2 Reutilización de código

10.3 protected

10.4 super

10.5 Override

# BLOQUE 11 — Polimorfismo y Clases Abstractas

11.1 Qué es el polimorfismo

11.2 Sobrescritura

11.3 Upcasting

11.4 Instanceof

11.5 Qué es abstract

11.6 Métodos abstractos

11.7 Cuándo usar clases abstractas

# BLOQUE 12 — Interfaces

13.1 Qué es una interfaz
13.2 implements
13.3 Diferencia entre clase abstracta e interfaz
13.4 Principio de diseño

# BLOQUE 13 — Colecciones

14.1 ArrayList
14.2 HashMap
14.3 Set
14.4 Iteradores

# BLOQUE 14 — Manejo de Errores

15.1 try-catch
15.2 Excepciones comunes
15.3 Crear excepciones personalizadas

# BLOQUE 15 — Proyecto Final

- Sistema de gestión de biblioteca
- Juego de combate por consola
- Simulador de cuentas bancarias
- Mini agenda de contactos

# TEMA 1 — Introducción a la Programación

## 1.1 ¿Qué es la programación?

**¿Qué significa programar?**

Programar es **dar instrucciones a un ordenador para que realice una tarea**.

Un ordenador no piensa.
No razona.
No interpreta intenciones.

Solo ejecuta instrucciones exactas, paso a paso.

Programar es aprender a comunicarse con la máquina de forma precisa.

**¿Qué es un algoritmo?**

Un algoritmo es:

Una secuencia ordenada de pasos para resolver un problema.

No necesita ser código.

Ejemplo sencillo:

Algoritmo para hacer un bocadillo

1. Tomar pan
2. Abrir el pan
3. Colocar el jamón
4. Cerrar el pan
5. Servir

Eso es un algoritmo.

Si cambias el orden, puede salir mal.
Si olvidas un paso, no funciona.

Exactamente igual ocurre en programación.

Características de un algoritmo

Un buen algoritmo debe ser:

- Claro
- Ordenado
- Finito (tiene principio y fin)
- Preciso

## 1.2¿Cómo piensa un ordenador?

Aquí viene algo muy importante.

Un ordenador no entiende:

- “más o menos”
- “hazlo rápido”
- “si puedes”

Solo entiende:

- Verdadero o falso
- Sí o no
- 1 o 0

Modelo básico: Entrada → Proceso → Salida

Todo programa funciona así:

### 1-Entrada (Input)

Información que recibe el sistema.

Ejemplo:

- Un número
- Un texto
- Un clic del usuario

### 2-Proceso

Lo que el programa hace con esa información.

Ejemplo:

- Sumar números
- Comparar valores
- Tomar decisiones

### 3-Salida (Output)

Resultado final.

Ejemplo:

- Mostrar un mensaje
- Guardar datos
- Imprimir un resultado

# 1.3 ¿Qué es Java?

Historia breve

Java fue creado en 1995 por Sun Microsystems.
Actualmente pertenece a Oracle.

Su lema principal era:

“Write once, run anywhere”

Es decir:
Escribes el programa una vez y puede ejecutarse en cualquier sistema que tenga la máquina virtual.

¿Qué es la JVM?

JVM significa:

Java Virtual Machine (Máquina Virtual de Java)

Es el sistema que permite que Java funcione en cualquier ordenador.

El proceso real es:

1. Escribes código Java (.java)
2. Se compila a bytecode (.class)
3. La JVM ejecuta ese bytecode

## Compilar vs Ejecutar

- Compilar

Convertir el código que escribes en un formato que la máquina entiende.

- Ejecutar

Hacer que el programa funcione realmente.

En Java:

Código → Compilador → Bytecode → JVM → Programa en ejecución

¿Por qué aprender Java?

Java es:

- Muy estructurado
- Muy claro
- Muy usado en empresas
- Base para entender bien la Programación Orientada a Objetos
- Utilizado en:
- Aplicaciones empresariales
- Android
- Sistemas bancarios
- Backend

Además, aprender Java te obliga a entender bien la lógica.
No es un lenguaje permisivo.
Eso es bueno para aprender correctamente.

**EJERCICIOS 1.1/1.2/1.3/1.4/1.5**

# TEMA 2 — Primer Contacto con Java

En este tema el alumno va a:

- Ver su primer programa real
- Entender cómo se ejecuta
- Perder el miedo al código
- Comprender la estructura básica de Java

Sin complicarlo todavía.

## 2.1 Instalación y Entorno de Trabajo

*(Esto puede adaptarse según si usas IntelliJ, Eclipse o VS Code)*

¿Qué necesitamos para programar en Java?

Para programar en Java hacen falta dos cosas:

1-JDK (Java Development Kit)

Es el paquete que incluye:

- El compilador
- La JVM
- Las herramientas necesarias para programar

Sin el JDK no se puede programar en Java.

2-Un IDE

IDE significa:

Integrated Development Environment

Es un programa que nos ayuda a:

- Escribir código
- Detectar errores
- Ejecutar el programa

Ejemplos comunes:

- IntelliJ IDEA
- Eclipse
- VS Code (con extensión)

El IDE no es obligatorio, pero facilita muchísimo el aprendizaje.

## 2.2Nuestro Primer Programa

Aquí empieza lo importante. Este es el programa más básico en Java:

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hola mundo");
    }
}
```

Vamos a desmontarlo pieza por pieza.

## 2.3 Anatomía del Primer Programa

*public class Main*

- `class` significa clase.
- En Java todo está dentro de una clase.
- El nombre del archivo debe llamarse exactamente igual que la clase.

Si la clase se llama `Main`, el archivo debe llamarse: Main.java

*public static void main(String[] args)*

Esta línea es especial.

Es el punto de entrada del programa.

Es donde empieza la ejecución.

Desglose simple:

- `public` → puede ejecutarse desde fuera
- `static` → pertenece a la clase
- `void` → no devuelve nada
- `main` → nombre obligatorio
- `(String[] args)` → permite recibir parámetros

Para un principiante basta con entender que el programa empieza aquí.

*System.out.println("Hola mundo");*

Esta línea hace algo visible.

- `System` → clase del sistema
- `out` → salida estándar
- `println` → imprimir línea (Imprime el texto en consola.)

## 2.4¿Qué ocurre cuando lo ejecutamos?

Proceso real:

1. Escribimos código
2. El compilador lo traduce
3. La JVM lo ejecuta
4. Se muestra el resultado en pantalla

## 2.5 Errores comunes en el primer contacto

Aquí es importante enseñar errores desde el principio.

❌Olvidar punto y coma

Java exige `;` al final de cada instrucción.

❌No cerrar llaves `{ }`

Las llaves delimitan bloques.

❌Escribir mal el nombre del archivo

Debe coincidir con la clase.

❌Confundir mayúsculas y minúsculas

Java distingue entre:

`main` ≠ `Main`

**<u>EJERCICIOS 2.1/2.2/2.3/2.4/2.5</u>**

# TEMA 3 — Variables y Tipos de Datos

Este tema es fundamental.
Si el alumno no entiende esto bien, todo lo demás se vuelve confuso.

## 3.1 ¿Qué es una variable?

Una variable es:

Un espacio en memoria donde guardamos información.

Piensa en una variable como una caja con una etiqueta.

Ejemplo mental:

Caja etiquetada como “edad”
Dentro guardamos: 33

En programación

Una variable tiene:

- Un tipo (qué tipo de dato puede guardar)
- Un nombre
- Un valor

En Java debemos declarar siempre el tipo.

## 3.2 Declarar una variable en Java

Ejemplo:

```
int edad = 33;
```

Esto significa:

- int → tipo de dato
- edad → nombre
- 33 → valor almacenado

También se puede hacer en dos pasos:

```
int edad;
edad = 33;
```

## 3.3 Tipos de Datos Primitivos

Java tiene varios tipos básicos.

### *int*

Números enteros.

int numero = 10;

int temperatura = -5;

### *double*

Números con decimales.

double altura = 1.75;

double precio = 19.99;

### *boolean*

Solo puede guardar:

- true
- false

boolean esMayor = true;

boolean estaActivo = false;

Muy importante para decisiones.

### *char*

Un solo carácter.

char letra = 'A';

Ojo:

- char usa comillas simples
- String usa comillas dobles

## 3.4 String (tipo no primitivo)

Sirve para texto.

String nombre = "Alvaro";

String ciudad = "Madrid";

Guarda palabras o frases.

## 3.5 Reglas para nombres de variables

Buenas prácticas:

✔ Empiezan con letra
✔ No usar espacios
✔ Usar camelCase

Ejemplo correcto:

*int numeroDeIntentos;*

## 3.6 Cambiar el valor de una variable

Las variables pueden modificarse.

*int edad = 33;*

*edad = 34;*

La variable mantiene el último valor asignado.

## 3.7 Operaciones con variables

Podemos usar operadores matemáticos.

*int a = 5;*

*int b = 3;*

*int suma = a + b;*

*int resta = a – b;*

*int multiplicacion = a * b;*

*int division = a / b;*

## 3.8 Mostrar variables en pantalla

*int edad = 33;*

*System.out.println(edad);*

También se puede combinar texto y variable:

*String nombre = "Alvaro";*

*System.out.println("Hola " + nombre);*

El símbolo + concatena texto.

## 3.9 Error común importante

Muchos principiantes confunden:

i*nt numero = 10;*

*numero = numero + 5;*

Esto significa:

Guardar en numero el valor anterior + 5.

No es matemáticas puras.
Es asignación.

**EJERCICIOS 3.1/3.2/3.3/3.4/3.5**

# TEMA 4 — Operadores y Comparaciones

En este tema el alumno aprende a:

- Realizar operaciones matemáticas
- Comparar valores
- Construir expresiones lógicas
- Prepararse para las estructuras condicionales

Este tema es el puente entre “guardar datos” y “pensar con lógica”.

## 4.1 Operadores Aritméticos

Sirven para hacer cálculos matemáticos.

Operadores básicos

| Operador | Significado |
|---|---|
| + | Suma |
| - | Resta |
| * | Multiplicación |
| / | División |
| % | Módulo (resto) |

## 4.2 Operadores de Asignación

Ya conocemos el básico:
int numero = 5;
Pero existen formas abreviadas:

| Operador | Equivale a |
|---|---|
| += | numero = numero + valor |
| -= | numero = numero - valor |
| *= | numero = numero * valor |
| /= | numero = numero / valor |

Ejemplo:

```
int puntos = 10;
puntos += 5;   // ahora vale 15
```

## 4.3 Operadores Relacionales (Comparación)

Sirven para comparar valores.

Devuelven siempre un boolean (true o false).

| Operador | Significado |
|---|---|
| == | Igual que |
| != | Distinto |
| > | Mayor que |
| < | Menor que |
| >= | Mayor o igual |
| <= | Menor o igual |

Ejemplo
int edad = 20;
System.out.println(edad >= 18);  // true
System.out.println(edad < 18);  // false

## 4.4 Operadores Lógicos

Sirven para combinar condiciones.

| Operador | Significado |
|---|---|
| && | Y lógico (AND) |
| ` | |
| ! | Negación (NOT) |

Ejemplo AND (&&)

```
int edad = 20;
boolean tieneEntrada = true;
System.out.println(edad >= 18 && tieneEntrada);
```

Resultado: true
Ambas condiciones deben cumplirse.

Ejemplo OR (||)

```
boolean esVip = false;
boolean tieneInvitacion = true;
System.out.println(esVip || tieneInvitacion);
```

Resultado: true
Con que una se cumpla, vale.

Ejemplo NOT (!)

```
boolean estaLloviendo = false;
System.out.println(!estaLloviendo);  // true
```

Invierte el valor.

## 4.5 Precedencia de Operadores

Java sigue reglas matemáticas.

Ejemplo:
int resultado = 5 + 3 * 2;
Multiplicación primero → 11

Si queremos cambiar el orden:
int resultado = (5 + 3) * 2;  // 16
Usar paréntesis mejora claridad y evita errores.

## 4.6 Expresiones Compuestas

Aquí empieza la lógica real.
Ejemplo:
int edad = 25;
boolean esEstudiante = true;
boolean descuento = edad < 30 && esEstudiante;
Aquí combinamos comparación + lógica.

# TEMA 5 — Estructuras Condicionales

Este tema es donde la programación deja de ser solo "guardar y operar datos" y empieza a pensar y decidir.

## 5.1 ¿Qué es una estructura condicional?

Una estructura condicional permite al programa ejecutar diferentes instrucciones según se cumpla o no una condición.
En la vida real sería algo así:
"Si llueve, me llevo paraguas; si no, salgo sin paraguas."

## 5.2 El condicional if

Sintaxis básica:

```
if (condición) {
    // instrucciones si la condición es verdadera
}
```

Ejemplo práctico:

```
int edad = 20;
if (edad >= 18) {
    System.out.println("Eres mayor de edad");
}
```

Solo imprime si la condición es true.

## 5.3 if…else

Cuando queremos manejar ambos casos (verdadero/falso):

```
int edad = 16;
if (edad >= 18) {
    System.out.println("Mayor de edad");
} else {
    System.out.println("Menor de edad");
}
```

## 5.4 if…else if…else

Cuando tenemos más de dos opciones:

```
int nota = 7;
if (nota >= 9) {
    System.out.println("Excelente");
} else if (nota >= 6) {
    System.out.println("Aprobado");
} else {
    System.out.println("Reprobado");
}
```

## 5.5 switch

Otra forma de tomar decisiones cuando tenemos muchas opciones basadas en un valor exacto:

```
int dia = 3;
switch (dia) {
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    case 3:
        System.out.println("Miércoles");
        break;
    default:
        System.out.println("Otro día");
}
```

- break evita que ejecute los siguientes casos
- default es lo que pasa si no coincide ningún caso

## 5.6 Comparaciones y operadores lógicos en condicionales

Se pueden usar operadores del Tema 4:

```
int edad = 20;
boolean tieneEntrada = true;
if (edad >= 18 && tieneEntrada) {
    System.out.println("Puede entrar al cine");
}
```

Aquí combinamos:

- Comparación (>=)
- Lógica (&&)

## 5.7 Buenas prácticas en condicionales

1. Siempre usar llaves { }, aunque sea una línea.
2. Evitar condicionales muy largas; dividir en variables auxiliares.
3. Usar nombres claros para las variables booleanas.

**EJERCICIOS**
**4.1/4.2/4.3/4.4/4.5**
**5.1/5.2/5.3/5.4/5.5**

# TEMA 6 — Bucles

Los bucles nos permiten ejecutar una acción varias veces sin tener que escribir el mismo código una y otra vez.
En la vida real sería algo así:
"Revisa tu correo cada 5 minutos hasta que encuentres un mensaje importante."

## 6.1 Concepto básico

Un bucle tiene:

1. Inicio → donde comienza la repetición
2. Condición → mientras sea verdadera, se repite
3. Cuerpo → instrucciones que se ejecutan cada vez
4. Actualización → normalmente cambiar algo para que la condición cambie

## 6.2 Bucle *for*

Se usa cuando sabemos cuántas veces queremos repetir.

Sintaxis:

```
for (inicialización; condición; actualización) {
    // instrucciones a repetir
}
```

Ejemplo:

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Hola " + i);
}
```

Salida:
Hola 1
Hola 2
Hola 3
Hola 4
Hola 5

- i = 1 → valor inicial
- i <= 5 → condición
- i++ → aumenta 1 cada vez

## 6.3 Bucle *while*

Se usa cuando no sabemos cuántas veces se repetirá, pero sí la condición.

Sintaxis:
```
while (condición) {
    // instrucciones
}
```

Ejemplo:
```
int contador = 1;
while (contador <= 5) {
    System.out.println("Hola " + contador);
    contador++;  // actualizar para evitar bucle infinito
}
```

## 6.4 Bucle *do-while*

Se parece al while, pero siempre ejecuta al menos una vez.

Sintaxis:
```
do {
    // instrucciones
} while (condición);
```

Ejemplo:
```
int contador = 1;
do {
    System.out.println("Hola " + contador);
    contador++;
} while (contador <= 5);
```

## 6.5 Control de bucles

### 6.5.1 break

Sirve para salir inmediatamente del bucle.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;  // termina el bucle
    }
    System.out.println(i);
}
```

Salida:
1
2
3
4

### 6.5.2 continue

Sirve para saltar la iteración actual y seguir con la siguiente.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;  // salta esta vuelta
    }
    System.out.println(i);
}
```

Salida:
1
2
4
5

## 6.6 Buenas prácticas

- Evitar bucles infinitos
- Usar nombres claros para variables de control (i, contador)
- Usar break y continue solo cuando sea necesario

**<u>EJERCICIOS 6.1/6.2/6.3/6.4/6.5</u>**

# TEMA 7 — Métodos (Funciones)

Un método es un bloque de código que realiza una tarea específica y que podemos usar varias veces sin repetirlo.

En la vida real sería algo así:

“Cada mañana preparo café: en vez de repetir los pasos cada día, tengo una rutina lista que solo ejecuto.”

## 7.1 ¿Por qué usar métodos?

Ventajas:

- Evitan repetir código
- Mejoran la claridad
- Permiten dividir un programa en tareas pequeñas
- Facilitan la corrección de errores

## 7.2Sintaxis básica de un método

```
public static void saludar() {
System.out.println("Hola a todos");
}
```

- public → accesible desde cualquier lugar
- static → pertenece a la clase (explicaremos más adelante “métodos de instancia”)
- void → no devuelve ningún valor
- saludar() → nombre del método
- { … } → contiene las instrucciones

## 7.3 Llamar a un método

Para ejecutar un método usamos su nombre seguido de paréntesis:

```
public class Main {
    public static void main(String[] args) {
        saludar();  // llama al método
    }
    public static void saludar() {
        System.out.println("Hola a todos");
    }
}
```

Salida:
Hola a todos

## 7.4 Métodos con parámetros

Podemos enviar información al método para que la use:

```
public static void saludarPersona(String nombre) {
    System.out.println("Hola " + nombre);
}
public static void main(String[] args) {
    saludarPersona("Alvaro");
    saludarPersona("Ana");
}
```

Salida:
Hola Alvaro
Hola Ana

- String nombre → parámetro del método
- Cada vez que llamamos al método, podemos pasar un valor distinto

## 7.5 Métodos con retorno

Algunos métodos devuelven un valor, en lugar de solo mostrar algo.

```
public static int sumar(int a, int b) {
    return a + b;
}
public static void main(String[] args) {
    int resultado = sumar(5, 3);
    System.out.println(resultado);
}
```

Salida:

8

- int → indica que devuelve un entero
- return → devuelve el valor al lugar donde se llamó el método

## 7.6 Métodos void vs métodos con retorno

| Tipo | Característica | Ejemplo |
|---|---|---|
| void | No devuelve nada | saludar() |
| retorna un valor | Devuelve un dato | sumar(a, b) |

## 7.7 Buenas prácticas

- Nombres descriptivos: sumar, calcularPromedio
- No hacer métodos demasiado largos
- Cada método debe cumplir una sola tarea

**<u>EJERCICIOS 7.1/7.2/7.3/7.4/7.5</u>**

# TEMA 8 — Arrays (Arreglos)

Los arrays son estructuras que nos permiten guardar varios datos del mismo tipo en una sola variable.
Piensa en un estante con varias cajas: cada caja puede guardar un valor y todas están numeradas (índice).

## 8.1Declaración y creación de arrays

1-Declarar un array sin asignarle valores

```
int[] numeros; // declaramos un array de enteros
numeros = new int[5]; // creamos espacio para 5 enteros
```

- int[] → tipo de datos del array
- new int[5] → reserva espacio para 5 elementos
- Los índices van de 0 a 4

2-Declarar y asignar valores directamente

```
int[] edades = {18, 20, 25, 30};
```

- Aquí ya tenemos 4 valores en el array
- edades[0] → 18
- edades[1] → 20
- edades[2] → 25
- edades[3] → 30

## 8.2Acceder a los elementos

```
System.out.println(edades[2]); // imprime 25
```

Importante: Si pides un índice que no existe, Java lanza un error (ArrayIndexOutOfBoundsException).

## 8.3Recorrer un array con bucles

Los bucles se usan mucho con arrays.

1-Bucle for tradicional

```
for (int i = 0; i < edades.length; i++) {
    System.out.println("Edad " + i + ": " + edades[i]);
}
```

- edades.length → número de elementos del array

2-Bucle for-each (simplificado)

```
for (int edad : edades) {
    System.out.println("Edad: " + edad);
}
```

- Itera automáticamente sobre todos los elementos
- Muy útil para leer arrays sin preocuparse por índices

## 8.4 Arrays multidimensionales

Un array puede tener filas y columnas, como una tabla.

```
int[][] matriz = {
    {1,2,3},
    {4,5,6},
    {7,8,9}
};
System.out.println(matriz[1][2]); // imprime 6
```

- Primer índice → fila
- Segundo índice → columna

## 8.5 Buenas prácticas con arrays

- Nombres claros: edades, numeros, precios
- No mezclar tipos de datos
- Evitar índices negativos o mayores que length - 1
- Usar bucles para recorrer y no repetir código

Punto pedagógico clave

- Los arrays son la base para listas y colecciones más avanzadas
- Combinar arrays + bucles + métodos permite crear programas más potentes y reutilizables

**EJERCICIOS 8.1/8.2/8.3/8.4/8.5**

# TEMA 9 — Colecciones en Java (ArrayList, HashMap, etc.)

Hasta ahora usamos arrays, pero tienen limitaciones:

- Tamaño fijo
- Poco flexibles
- Métodos muy básicos

Las colecciones solucionan eso.

## 9.1 ¿Qué es el Framework de Colecciones?

Es un conjunto de clases e interfaces dentro de:
java.util
Las más importantes para empezar:

- ArrayList
- LinkedList
- HashSet
- HashMap

Hoy empezamos por las más usadas en el mundo real:
***ArrayList y HashMap***

## 9.2 ArrayList

Es como un array, pero:
✔ Tamaño dinámico
✔ Métodos útiles
✔ Más flexible

### Declaración

```
import java.util.ArrayList;
ArrayList<String> nombres = new ArrayList<>();
<String> indica el tipo de dato que almacenará (esto se llama
genéricos).
```

### Métodos principales

```
nombres.add("Ana");
nombres.add("Luis");
nombres.add("Carlos");
System.out.println(nombres.get(0)); // Ana
nombres.remove("Luis");
System.out.println(nombres.size()); // tamaño
```

## Recorrer un ArrayList

*Con for tradicional:*

```
for (int i = 0; i < nombres.size(); i++) {
    System.out.println(nombres.get(i));
}
```

*Con for-each:*

```
for (String nombre : nombres) {
   System.out.println(nombre);
}
```

## Diferencia Array vs ArrayList

| Array | ArrayList |
|---|---|
| Tamaño fijo | Dinámico |
| Más rápido | Más flexible |
| Sintaxis simple | Más métodos útiles |

# 9.3 HashMap

Un HashMap guarda datos en formato:

clave → valor

Como un diccionario.

## Declaración

```
import java.util.HashMap;
HashMap<String, Integer> edades = new HashMap<>();
```

Aquí:

- Clave = String
- Valor = Integer

## Uso básico

```
edades.put("Ana", 25);
edades.put("Luis", 30);
System.out.println(edades.get("Ana")); // 25
```

## Recorrer un HashMap

```
for (String clave : edades.keySet()) {
    System.out.println(clave + " tiene " + edades.get(clave) + "
años");
}
```

## 9.4 ¿Cuándo usar cada uno?

| Situación | Estructura ideal |
|---|---|
| Lista ordenada de elementos | ArrayList |
| Buscar por clave específica | HashMap |
| Evitar duplicados | HashSet |
| Lista con muchas inserciones intermedias | LinkedList |

## Concepto importante: Genéricos

```
ArrayList<Integer>
HashMap<String, Persona>
Los < > indican el tipo de dato.
```

Evitan errores en tiempo de ejecución.

Ejemplo práctico combinando POO + Colecciones

```
import java.util.ArrayList;
public class Main {
    public static void main(String[] args) {
        ArrayList<String> jugadores = new ArrayList<>();
        jugadores.add("Messi");
        jugadores.add("Ronaldo");
        jugadores.add("Mbappé");
        for (String jugador : jugadores) {
            System.out.println(jugador);
        }
    }
}
```

# TEMA 10 — Introducción a la Programación Orientada a Objetos (POO)

## 10.1 ¿Qué es la POO?

La POO es un paradigma de programación basado en:

- Objetos → representan cosas o conceptos del mundo real
- Clases → "planos" o "moldes" para crear objetos

Ejemplo del mundo real:
Una clase Coche define cómo es un coche.
Cada coche que fabriquemos con esa clase es un objeto diferente.

## 10.2 Diferencias con la programación estructurada

| Característica | Estructurada | Orientada a Objetos |
|---|---|---|
| Organización | Secuencia de instrucciones | Clases y objetos |
| Reutilización | Funciones sueltas | Herencia y métodos de objetos |
| Datos | Variables sueltas | Atributos de objetos |
| Lógica | Funciones + condicionales | Métodos que actúan sobre datos |

## 10.3 Qué es una clase

Una clase es un molde que define:

- Atributos → características o propiedades
- Métodos → acciones o comportamientos

Ejemplo:

```
public class Coche {
    // atributos
    String color;
    String marca;
    int velocidad;
    // método
    public void acelerar() {
        velocidad += 10;
        System.out.println("Velocidad: " + velocidad);
    }
}
```

- Coche → clase
- color, marca, velocidad → atributos
- acelerar() → método que modifica el estado del objeto

## 10.4 Qué es un objeto

Un objeto es una instancia de una clase.

```
public class Main {
    public static void main(String[] args) {
        Coche miCoche = new Coche(); // creamos un objeto
        miCoche.color = "Rojo";
        miCoche.marca = "Toyota";
        miCoche.acelerar();  // llama al método
    }
}
```

- miCoche es un objeto de la clase Coche
- Cada objeto puede tener valores diferentes en sus atributos

## 10.5 Constructores

Un constructor es un método especial que se llama automáticamente al crear un objeto.

```
public class Coche {
    String color;
    String marca;
    int velocidad;
    // constructor
    public Coche(String c, String m) {
        color = c;
        marca = m;
        velocidad = 0;
    }
    public void acelerar() {
        velocidad += 10;
        System.out.println("Velocidad: " + velocidad);
    }
}
public class Main {
    public static void main(String[] args) {
        Coche miCoche = new Coche("Azul", "Honda");
        miCoche.acelerar();
    }
}
```

- Permite inicializar atributos desde el principio
- Facilita crear objetos únicos con propiedades diferentes

**<u>EJERCICIOS 9.1/9.2/9.3/9.4/9.5</u>**

# TEMA 11 — Encapsulamiento y Herencia

## 11.1 Encapsulamiento

El encapsulamiento consiste en:
Proteger los datos de una clase y controlar cómo se accede a ellos.
Hasta ahora hacíamos esto:

`miCoche.velocidad = 100;` // ✕ cualquier parte del programa puede cambiarla

Eso no es buena práctica.

La solución: Modificadores de acceso
Java tiene 4:

- public → accesible desde cualquier parte
- private → solo accesible dentro de la clase
- protected
- (default) → solo dentro del mismo paquete

El más importante ahora es:

### *private*

Ejemplo con encapsulamiento

```
public class Coche {
    private String marca;
    private int velocidad;
    public Coche(String marca) {
        this.marca = marca;
        this.velocidad = 0;
    }
    public void acelerar() {
        velocidad += 10;
    }
    public int getVelocidad() {
        return velocidad;
    }
    public String getMarca() {
        return marca;
    }
}
```

¿Qué cambia?

- Ahora nadie puede hacer coche.velocidad = 200;
- Solo se puede modificar a través de métodos

## Getters y Setters

Son métodos que:

- Get → devuelven el valor
- Set → modifican el valor (si queremos permitirlo)

Ejemplo:

```
public void setVelocidad(int velocidad) {
    if (velocidad >= 0) {
        this.velocidad = velocidad;
    }
}
```

Aquí estamos controlando que no pueda ser negativa.

# 11.2 Herencia

La herencia permite que una clase:

Herede atributos y métodos de otra clase.

Se usa la palabra clave:

## *extends*

Ejemplo base

Clase padre:

```
public class Vehiculo {
    protected String marca;
    public Vehiculo(String marca) {
        this.marca = marca;
    }
    public void arrancar() {
        System.out.println("El vehículo arrancó");
    }
}
```

Clase hija:

```
public class Coche extends Vehiculo {
    private int puertas;
    public Coche(String marca, int puertas) {
        super(marca);
        this.puertas = puertas;
    }
    public void mostrarInfo() {
        System.out.println("Marca: " + marca);
        System.out.println("Puertas: " + puertas);
    }
}
```

## Conceptos clave

- extends → indica herencia
- super() → llama al constructor del padre
- La clase hija hereda métodos públicos y protegidos

## Ventajas de la herencia

✔ Reutilización de código
✔ Mejor organización
✔ Permite crear jerarquías lógicas

Ejemplo real:
Vehiculo
↓
Coche
↓
CocheElectrico

**EJERCICIOS 11.1/11.2/11.3/11.4/11.5**

# TEMA 12 — Polimorfismo y Clases Abstractas

Aquí es donde Java pasa de “organizado” a “elegante”.

## 12.1 ¿Qué es el Polimorfismo?

Polimorfismo significa:
Un mismo método puede comportarse de diferentes maneras según el objeto que lo use.

“Poli” = muchos
“Morfo” = formas
“Muchas formas”

Ejemplo básico

Clase padre:
```
public class Animal {
    public void hacerSonido() {
        System.out.println("El animal hace un sonido");
    }
}
```

Clase hija 1:
```
public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Guau");
    }
}
```

Clase hija 2:
```
public class Gato extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Miau");
    }
}
```

Uso en main
```
public class Main {
    public static void main(String[] args) {
        Animal a1 = new Perro();
        Animal a2 = new Gato();
        a1.hacerSonido(); // Guau
        a2.hacerSonido(); // Miau
    }
}
```

Observa esto:

Animal a1 = new Perro();
El tipo es Animal, pero el comportamiento es Perro.
Eso es polimorfismo.

## 12.2 Sobrescritura (Override)

Cuando una clase hija redefine un método del padre:

```
@Override
public void hacerSonido() {
```

- @Override no es obligatorio
- Pero ayuda al compilador a detectar errores

## 12.3 Clases Abstractas

A veces queremos una clase que:
- Sirva como modelo
- Pero no tenga sentido crear objetos directamente

Ejemplo: “Animal” en general.
No existe “animal genérico” en la vida real.

### Clase abstracta

```
public abstract class Animal {
    public abstract void hacerSonido();
    public void dormir() {
        System.out.println("El animal está durmiendo");
    }
}
```

¿Qué significa esto?
- abstract → no se puede crear con new
- Un método abstracto no tiene cuerpo
- Las clases hijas están obligadas a implementarlo

### Implementación obligatoria

```
public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Guau");
    }
}
```

Si no implementas el método → error de compilación.

## Ventajas de clases abstractas

✔ Obligan a mantener una estructura común
✔ Sirven como contrato base
✔ Mejor diseño de jerarquías

## 12.4 Diferencia entre clase normal y abstracta

| Clase normal | Clase abstracta |
|---|---|
| Puede crear objetos | No puede crear objetos |
| Métodos completos | Puede tener métodos abstractos |
| Opcional heredar | Pensada para heredar |

## Concepto importante: Polimorfismo + Herencia

Esto permite hacer cosas como:

```
Animal[] animales = new Animal[3];
animales[0] = new Perro();
animales[1] = new Gato();
animales[2] = new Perro();
for (Animal a : animales) {
    a.hacerSonido();
}
```

El bucle no sabe qué tipo concreto es cada animal.
Java decide automáticamente qué método ejecutar.
Eso es programación profesional.

**EJERCICIOS 12.1/12.2/12.3/12.4/12.5**

# TEMA 13 — Interfaces

Si las clases abstractas eran un “modelo base”,
las interfaces son un contrato obligatorio.

## 13.1 ¿Qué es una interfaz?

Una interfaz es:

Un contrato que obliga a una clase a implementar ciertos métodos.
No define “cómo” se hacen las cosas,
solo define “qué” debe existir.

### Sintaxis básica

```
public interface Volador {
    void volar();
}
```

Observa:

- No hay public delante del método (es implícito)
- No hay cuerpo { }
- No hay atributos normales (solo constantes)

## 13.2 Implementar una interfaz

Para usar una interfaz se usa:

### *implements*

Ejemplo:

```
public class Pajaro implements Volador {
    @Override
    public void volar() {
        System.out.println("El pájaro está volando");
    }
}
```

Si no implementas volar() → error de compilación.

## 13.3 Diferencia entre extends y implements

| Palabra clave | Se usa para |
|---|---|
| extends | Heredar de una clase |
| implements | Implementar una interfaz |

## 13.4 ¿Por qué usar interfaces?

Imagina que tienes:

- Pajaro
- Avion
- Superman

No son el mismo tipo de objeto.
No comparten una clase padre lógica.
Pero todos pueden volar.
Ahí usamos una interfaz.

Ejemplo completo

```
public interface Volador {
    void volar();
}
public class Avion implements Volador {
    @Override
    public void volar() {
        System.out.println("El avión despega");
    }
}
public class Superman implements Volador {
    @Override
    public void volar() {
        System.out.println("Superman vuela a toda velocidad");
    }
}
```

## Polimorfismo con interfaces

```
public class Main {
    public static void main(String[] args) {
        Volador v1 = new Avion();
        Volador v2 = new Superman();
        v1.volar();
        v2.volar();
    }
}
```

Observa:
El tipo es Volador,
pero el comportamiento cambia según el objeto real.
Eso es diseño profesional.

## 13.5 Diferencias: Clase Abstracta vs Interfaz

| Clase Abstracta | Interfaz |
|---|---|
| Puede tener atributos normales | Solo constantes |
| Puede tener métodos con código | Métodos sin cuerpo (normalmente) |
| Se usa extends | Se usa implements |
| Solo puede extender una | Puede implementar varias |

## 13.6 Ventaja clave

Una clase puede implementar varias interfaces:

```
public class Robot implements Volador, Nadador, Disparador {
```

Pero solo puede extender una clase.
Por eso en arquitectura profesional se usan muchísimas interfaces.

**EJERCICIOS 13.1/13.2/13.3/13.4/13.5**

# TEMA 14 — Manejo de Excepciones (try-catch)

Hasta ahora, si ocurre un error, el programa se detiene.
Ejemplo clásico:

int resultado = 10 / 0;

Error en tiempo de ejecución:

***ArithmeticException***

Aquí es donde entran las excepciones.

## 14.1 ¿Qué es una excepción?

Una excepción es:
Un error que ocurre durante la ejecución del programa.
Ejemplos comunes:

- Dividir entre 0
- Acceder a un índice inexistente en un array
- Convertir texto a número incorrectamente
- Leer un archivo que no existe

## 14.2 try – catch

Sirve para "atrapar" el error y evitar que el programa se detenga.

Sintaxis básica

```
try {
    // código que puede fallar
} catch (TipoDeExcepcion e) {
    // qué hacer si ocurre el error
}
```

### Ejemplo 1 — División por cero

```
public class Main {
    public static void main(String[] args) {
        try {
            int resultado = 10 / 0;
            System.out.println(resultado);
        } catch (ArithmeticException e) {
            System.out.println("No se puede dividir entre cero.");
        }
        System.out.println("El programa continúa...");
    }
}
```

Salida:
No se puede dividir entre cero.

El programa continúa...
El programa no se detiene.

## 14.3 Excepciones comunes en Java

| Excepción | Cuándo ocurre |
|---|---|
| ArithmeticException | División por cero |
| ArrayIndexOutOfBoundsException | Índice inválido en array |
| NullPointerException | Objeto null |
| NumberFormatException | Conversión inválida de texto a número |

## Ejemplo 2 — Índice inválido

```
try {
    int[] numeros = {1, 2, 3};
    System.out.println(numeros[5]);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Índice fuera de rango.");
}
```

## 14.4 Múltiples catch

```
try {
    int numero = Integer.parseInt("abc");
    int resultado = 10 / numero;
} catch (NumberFormatException e) {
    System.out.println("No es un número válido.");
} catch (ArithmeticException e) {
    System.out.println("Error matemático.");
}
```

Java evalúa los catch en orden.

## 14.5 finally

El bloque finally siempre se ejecuta.

```
try {
    System.out.println("Intentando...");
} catch (Exception e) {
    System.out.println("Error.");
} finally {
    System.out.println("Esto siempre se ejecuta.");
}
```

Se usa mucho para:

- Cerrar archivos
- Cerrar conexiones
- Liberar recursos

## 14.6 throw (lanzar excepción manualmente)

Puedes provocar una excepción tú mismo.

```
public static void validarEdad(int edad) {
    if (edad < 18) {
        throw new IllegalArgumentException("Edad no válida");
    }
}
```

**EJERCICIOS 14.1/14.2/14.3/14.4/14.5**